SIMATS ENGINEERING

ASSESSMENT TOOL 2: SCENARIO-BASED

Cloud Computing and Big Data Analytics

Course Code: CSA15 | Course Outcome: CO2

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments (BL3).

Assessment Tool Weightage: 20% Total Marks: 25 Question Count: 5

Code of Conduct

I Bandaru sai ganesh(1925260178) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Bandaru sai ganesh

Assessment Tasks: Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning. Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage
Understanding of Scenario	20%
Identification of Risks / Issues	20%
Application of Concepts	25%
Decision Justification	20%
Clarity	15%

Answer 1: Cloud Architecture for Video Streaming

Question: A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

1. Proposed architecture

Use a multi-region cloud architecture with a content delivery network (CDN) in front of the streaming platform. Store video files in durable object storage. Use adaptive bitrate streaming (such as HLS or MPEG-DASH) so each viewer receives a suitable video quality based on available bandwidth. Keep application APIs and user-management services behind load balancers, and use managed databases and queues where appropriate.

2. Scalability and caching

Cache video segments and frequently requested content at CDN edge locations close to users. This reduces latency and origin-server traffic. Use horizontal scaling to add application instances as demand rises. Separate stateless services from persistent data so additional instances can be added without disrupting sessions. Cache suitable metadata and catalogue responses, while applying expiry and invalidation rules to prevent stale content.

3. Load balancing and auto-scaling

A load balancer distributes incoming requests across healthy application instances and performs health checks. Auto-scaling policies can add instances when CPU utilization, request rate, queue length, or latency crosses a threshold, and remove them when demand falls. Configure minimum capacity and scheduled scaling for predictable events, and test scaling limits before major launches. The CDN, adaptive bitrate streaming, and sufficient origin capacity work together to reduce buffering.

4. Cost optimization and conclusion

Use CDN caching to reduce repeated origin downloads, lifecycle policies to move older videos to lower-cost storage, and autoscaling to avoid paying for idle compute. Select appropriate storage classes and data-transfer options, monitor cost by service, and use reserved or committed-use discounts for stable baseline demand. Apply budgets and alerts. The design balances performance, resilience, and cost.

Answer 2: Consistent Multi-Cloud Container Deployment

Question: A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

1. Proposed solution

Package each application as a versioned container image and store it in a secured container registry. Use Kubernetes or a managed Kubernetes service in each cloud to deploy and operate containers. Keep deployment manifests, Helm charts, configuration templates, and infrastructure definitions in version control. A CI/CD pipeline should build, test, scan, sign, and promote the same image digest through development, staging, and production.

2. Consistency and orchestration

Use declarative manifests and GitOps tools so the desired application state is version-controlled and reconciled automatically in each cluster. Standardize supported Kubernetes versions, resource requests and limits, secrets handling, logging, and health probes. Use rolling updates or canary releases with rollback when health checks fail. Avoid cloud-specific dependencies where portability is required, while documenting unavoidable provider differences.

3. Service discovery and scaling

Use Kubernetes Services and cluster DNS for service discovery within each cluster. For cross-cloud communication, use a secured service mesh, global traffic manager, or API gateway with explicit routing and mutual TLS where suitable. Configure horizontal pod autoscaling based on CPU, memory, or application metrics, and cluster autoscaling to add worker nodes. Set resource quotas and availability rules to prevent noisy-neighbour problems.

4. Benefits, risks, and conclusion

Multi-cloud can improve resilience, reduce dependence on one provider, and support regional or regulatory needs. Risks include operational complexity, different IAM and networking models, data-transfer charges, inconsistent managed services, and more difficult troubleshooting. Reduce these risks with common automation, centralized observability, tested recovery procedures, least-privilege access, and clear ownership. Multi-cloud is useful when its resilience or business benefits justify its extra complexity.

Answer 3: Cloud Backup and Disaster Recovery

Question: An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning. Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

1. Backup and protection design

Enable object versioning and, where supported, soft-delete or recovery windows for important cloud data. Schedule automated, encrypted backups of databases, file stores, configurations, and application state. Keep backup copies in a separate account or subscription and, for critical systems, a separate region. Use immutable or write-protected backup retention to reduce the risk of accidental or malicious deletion.

2. Replication and recovery objectives

Replication maintains a copy of data in another location, but replication alone may copy accidental deletions or corruption; therefore, retain independent point-in-time backups. Recovery Point Objective (RPO) is the maximum acceptable amount of data loss measured in time. Frequent snapshots or continuous log replication reduce RPO. Recovery Time Objective (RTO) is the target time to restore service; pre-provisioned recovery infrastructure, automation, and documented runbooks help meet it.

3. Recovery procedure

After an incident, declare the recovery event, identify the last known-good recovery point, and prevent damaged data from overwriting backups. Restore data into an isolated environment, validate integrity and application dependencies, then redirect users when checks pass. Prioritize essential learning services and student records. Communicate status to users and stakeholders, and retain incident logs for review.

4. Business continuity and conclusion

Define owners, contact lists, recovery priorities, and manual workarounds. Test restores regularly and run disaster-recovery exercises to measure actual RTO and RPO. Monitor backup job completion and alert on failures, unusual deletions, or retention-policy changes. A combination of versioning, isolated backups, replication, and tested restoration provides stronger protection than any single mechanism.

Answer 4: Secure Hybrid Cloud for a Government Agency

Question: A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

1. Proposed hybrid architecture

Keep regulated or highly sensitive applications and datasets on-premises when required by policy. Place suitable public-facing services and non-sensitive workloads in the public cloud. Connect the environments using a private dedicated connection or site-to-site VPN, with redundant links where availability is important. Use network segmentation, firewalls, private endpoints, and tightly controlled routes so only approved systems can communicate.

2. Identity and access management

Integrate on-premises identity with cloud identity using a supported federation standard such as SAML or OpenID Connect. Apply single sign-on where appropriate, multi-factor authentication, role-based access control, and least privilege. Use separate administrative accounts, privileged-access controls, short-lived credentials, and centralized audit logs. Review access regularly and remove permissions when roles change.

3. Encryption and data protection

Encrypt data in transit using TLS and encrypt stored data using approved encryption services. Manage keys securely through a controlled key-management system, restrict key administrators, and rotate keys according to policy. Classify data before moving it to the cloud, apply data-loss prevention where suitable, and ensure backups and logs follow the same compliance requirements.

4. Challenges and conclusion

Hybrid cloud introduces complexity in networking, identity synchronization, monitoring, patching, latency, and policy enforcement. Different tools can create visibility gaps, and connectivity failures can interrupt dependent applications. Address these issues with common security policies, centralized logging and alerting, configuration automation, documented ownership, redundancy, and regular compliance reviews. The design allows cloud flexibility while keeping restricted workloads in the required environment.

Answer 5: Multi-Region High Availability for SaaS

Question: A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

1. Multi-region deployment strategy

Deploy application services in at least two geographically separate cloud regions. Place each region across multiple availability zones where available. Use a global DNS or traffic-management service to direct users to a healthy region, and deploy identical, tested application versions using infrastructure as code. A warm-standby design can reduce cost; active-active can improve capacity and recovery speed but requires more careful data consistency management.

2. Data replication and failover

Replicate data to the recovery region using the database's supported cross-region replication mechanism. Choose synchronous or asynchronous replication according to latency, consistency, RPO, and cost requirements. With asynchronous replication, some recent writes may be lost during a regional failure, so document the achievable RPO. Use health checks and automated or operator-approved failover, and prevent split-brain writes by ensuring only the intended region is authoritative.

3. Monitoring and alerting

Collect metrics, logs, and traces from both regions. Monitor request latency, error rate, traffic, database replication lag, storage capacity, and health-check status. Configure alerts with meaningful thresholds and route them to an on-call team. Use synthetic transactions to verify the service from outside the cloud environment, not only from inside each region.

4. Testing and conclusion

Regularly test regional failover and failback, including DNS or traffic propagation, data integrity, credentials, quotas, and dependencies. Maintain runbooks and communicate incident status. Measure recovery against defined RTO and RPO targets. Multi-region deployment improves availability only when replication, failover, monitoring, and recovery procedures are all tested.